

Module : MATHEMATIQUES ET INITIATION A L'INFORMATIQUE

Elément: "Initiation à l'informatique"

Elément: "Initiation à l'informatique"



Initiation à l'informatique

❑ Buts et motivation (1)

L'objectif de ce cours est de présenter les concepts fondamentaux, le "comment ça marche, l'informatique?".

Le cours est divisé en trois parties:

1ère partie: rassemble les concepts essentiels sur les notions d'**informatique**, d'**ordinateur**, de **système d'exploitation**.

Deuxième partie: introduit le concept d'**algorithmique** qui permet de décrire de façon simple et concise le fonctionnement d'un **programme**.

Troisième partie: introduit les concepts de **langage de programmation**, de programme informatique et décrit comment se déroule son exécution.

Initiation à l'informatique

❑ But et motivation (2)

Être capable de :

- ❑ Comprendre un algorithme et expliquer ce qu'il fait.
- ❑ Savoir transcrire les différentes étapes de résolution d'un problème sous forme d'algorithme.
- ❑ Construire un algorithme simple, le programmer et comprendre comment se déroule son exécution.

- ❑ Modifier un algorithme existant pour obtenir des résultats améliorés.
- ❑ Développer des programmes informatiques qui soient:
 - ✓ Simples
 - ✓ Correctes
 - ✓ Efficaces
 - ✓ Rapides

Plan de cours

1 Généralité sur l'informatique

Définitions: Informatique, Données, Ordinateur, Système d'exploitation

2 Algorithmique

Chapitre 1: Les éléments de base d'un algorithme

I- Introduction

II- Structure générale d'un algorithme

III- Les variables et les constantes

IV- Les instructions de base

Chapitre 2: Les structures conditionnelles et répétitives

I- Les structures conditionnelles (alternatives)

II- Les structures répétitives (Les boucles)

III- L'organigramme

3 Base de programmation

Notions de base en programmation avec Quick Basic

Partie 1: Généralité sur l'informatique

Généralité sur l'informatique

□ Définition

D'où vient le mot **informatique**?

le mot informatique fait sa première apparition en 1957 sous la plume d'un ingénieur allemand, nommé **Karl Steinbuch**. Ce dernier, est un auteur d'un ouvrage sur le traitement automatisé de données.

Informatique = **Information** + **Automatique**

→ **Définition:** *L'informatique est la science du traitement automatique de l'information.*



Généralité sur l'informatique

□ Personnalités



Charles Babbage: Il fut le premier à énoncer l'idée et le principe d'un ordinateur.



Steve Jobs: Cofondateur de la société APPLE, il a inventé l'interface graphique des systèmes d'exploitation. Il a également révolutionné le monde mobile grâce notamment au iPhone.



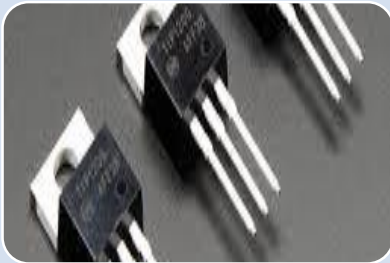
Bill Gates: Il est le fondateur de Microsoft. Il a inventé le système d'exploitation le plus utilisé de nos jours (Microsoft Windows).



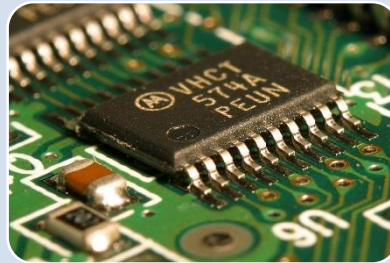
Tim Bernes Lee: est un informaticien britannique, celui qui a inventé le www qui est à la base de l'internet que nous connaissons aujourd'hui.

Généralité sur l'informatique

❑ Essor



Invention du transistor (1947):
L'invention du transistor.



•**1958:** C'est l'invention du circuit intégré.



Invention du microprocesseur: En 1969, E. Hoff, ingénieur à Intel inventa le premier microprocesseur Intel 4004 commercialisé, le 15 novembre 1971.



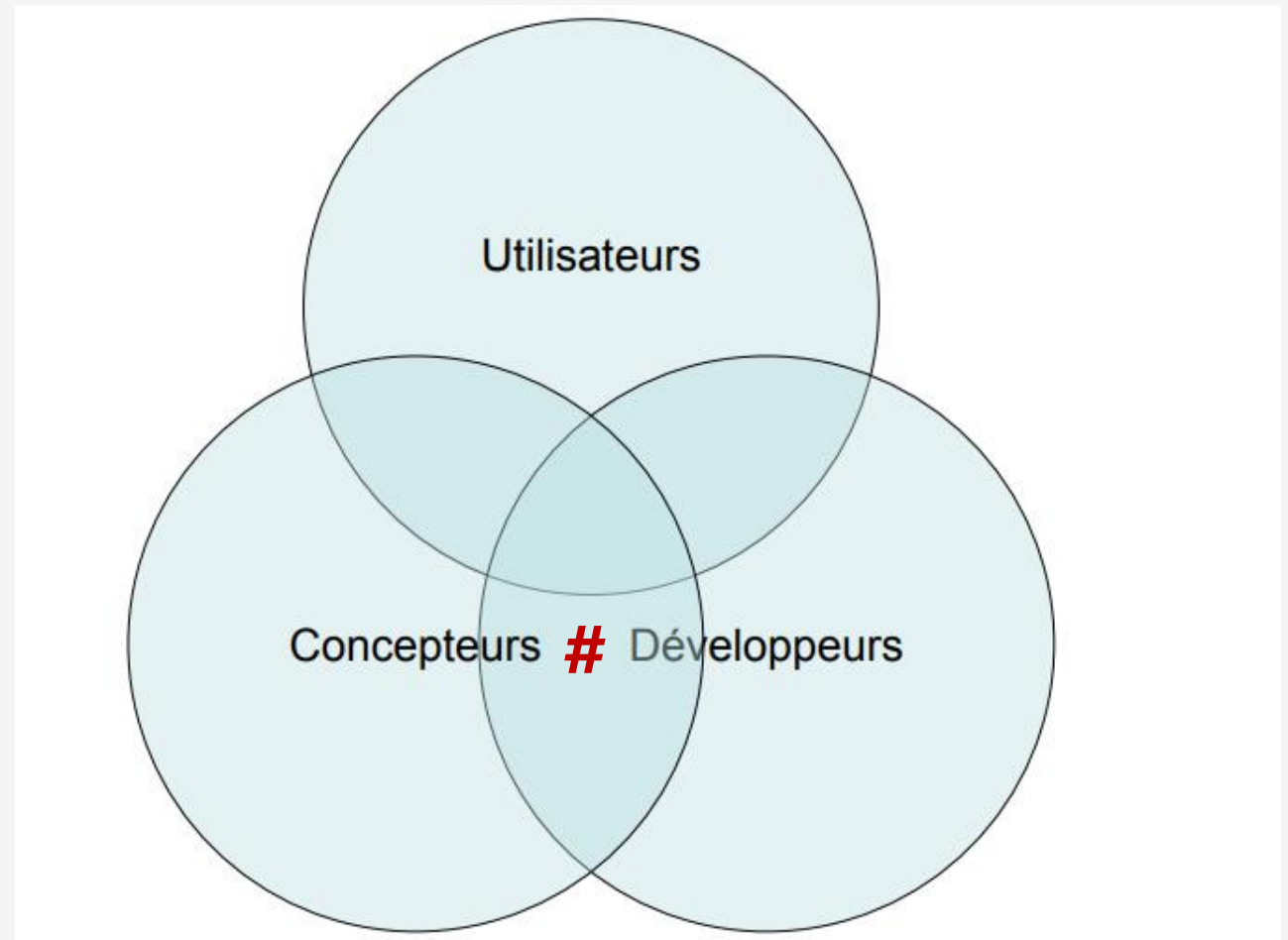
L'ordinateur personnel (PC): c'est un ordinateur destiné à l'usage d'une personne, de prix accessible et dont les dimensions sont assez réduites pour tenir sur un bureau. La première machine appelée micro-ordinateur est le Micral N, breveté en 1973 par le Français François Gernelle.

ont permis de nouvelles possibilités de miniaturisation et de puissance de calcul.

Généralité sur l'informatique

□ Usagers de l'informatique

- **Concepteurs:** un concepteur est une personne qui est chargée d'identifier les besoins des utilisateurs et de les spécifier. Son rôle consiste en particulier à expliquer les concepts à des experts non informaticiens.
- **Développeurs:** un développeur (programmeur) est un informaticien qui réalise des logiciels et les met en œuvre à l'aide de langages de programmation.
- **Utilisateurs:** Une personne qui utilise un système informatisé (ordinateur ou robot) mais qui n'est pas nécessairement informaticien.



Généralité sur l'informatique

❑ Technologie de l'information

Formats de données ??

- 
- 1 Nombres ✓ prix, poids, volume, température, vitesse, pression...
 - 2 Textes ✓ courrier, publications, articles de presse...
 - 3 Images ✓ plans, dessins, graphiques, diagrammes, cartes géographiques, photos, images 3D...
 - 4 Sons ✓ paroles, audios, bruitages ou musique.
 - 5 Vidéo ✓ prises de vue, clips ou films.

Généralité sur l'informatique

❑ Données / Informations

- Il existe une différence subtile entre les données et les informations. Les données sont les faits ou les détails à partir desquels l'information **est dérivée**.
- Les données individuelles sont rarement utiles seules. Pour que les données deviennent des informations, elles doivent être **mises en contexte**.

❑ Donnée :

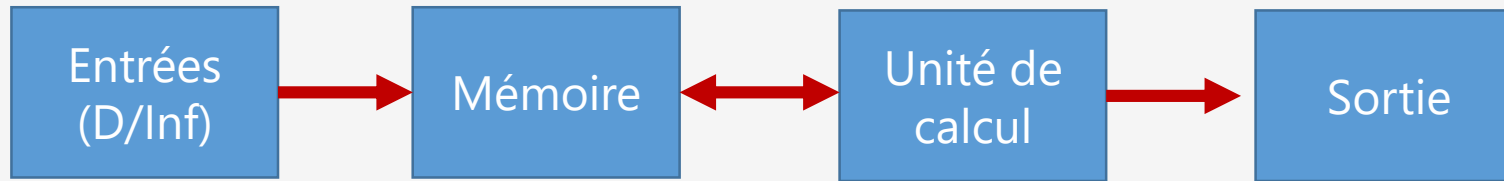
- Les données sont des faits bruts et non organisés qui doivent être traités.
- Les données sont toujours interprétées, par un homme ou une machine, pour en tirer un sens.
- Les données contiennent des chiffres, des énoncés et des caractères sous forme brute.

❑ Information :

- Lorsque des données sont **traitées, organisées, structurées** ou présentées dans un contexte donné afin de les rendre utiles, on les appelle des **informations**.
- Lorsque les données sont transformées en informations, elles ne contiennent jamais de détails inutiles.

Généralité sur l'informatique

□ Système de traitement de l'information



- Un système de traitement de l'information est composé de quatre unités:
- l'unité d'entrée (anglais: input), qui permet de faire entrer les informations dans le système. (l'entrée des informations utilise notamment des procédés de numérisation)
 - l'unité de stockage (anglais: storage) qui permet de conserver les informations.
 - l'unité de traitement (anglais: processor) qui comme son nom l'indique va traiter les informations.
 - l'unité de sortie (anglais: output) qui permet de faire sortir les résultats des traitements.

Généralité sur l'informatique

□ Représentation et stockage des données

- Elles sont de toutes sortes (texte, image, son, video), mais sont numérisées sous forme de 0 et de 1 (codées en binaire), on parle de **codage de l'information**.
 - **Le codage d'une information** consiste à établir une **correspondance** entre la représentation externe (habituelle) de l'information (le caractère A ou le nombre 36 par exemple), et sa représentation interne dans la machine, **qui est une suite de bits**.
 - On utilise la représentation **binaire** car:
 1. les données binaires sont plus faciles à mémoriser sur des supports physiques (bandes magnétiques, disques, usb, etc.)
 2. La représentation binaire est facile à réaliser techniquement à l'aide de bistables (système à deux états réalisés à l'aide de transistors).
- Par exemple, si l'on veut stocker un nombre entier sur le disque dur d'un ordinateur, on code généralement ce nombre en base 2 (langage machine) $(10)_2$ au lieu de le coder en base 10 (système décimal).

Exemple: le nombre 12 (en base 10) sera codé en base 2 par la suite binaire **00001100**,

Généralité sur l'informatique

□ Représentation et stockage des données

- On mesure la quantité de mémoire stockée dans les ordinateurs en :
 - Octet : 8 bits → permet de différencier **256** informations (ex : caractères)
 - Kilo Octet (Ko) : **2¹⁰** octets = 1024 octets
 - Mega Octet (Mo) : **2²⁰** octets = 1024 Kilo octets = 1 million d'octets
 - Giga Octet (Go) : **2³⁰** octets = 1 073 741 824 octets = **1024** Mo = 1 milliard d'octets
 - Tera Octet, Peta Octet, Exa Octet, zeta etc....

Généralité sur l'informatique

❑ Architecture de base d'un ordinateur

Définition: *Un ordinateur est une machine de traitement de l'information. Il est capable **d'acquérir** de l'information, de la **stocker**, de la **transformer** en effectuant des traitements quelconques, puis de la **restituer** sous une autre forme.*

❑ Principe de fonctionnement

Les deux principaux constituants d'un ordinateur sont **la mémoire principale** et le **processeur**. La mémoire principale (MP) permet de stocker de l'information, (**programmes** + données) tandis que le processeur exécute pas à pas les **instructions** composant les programmes.

Généralité sur l'informatique

❑ Notion de programme

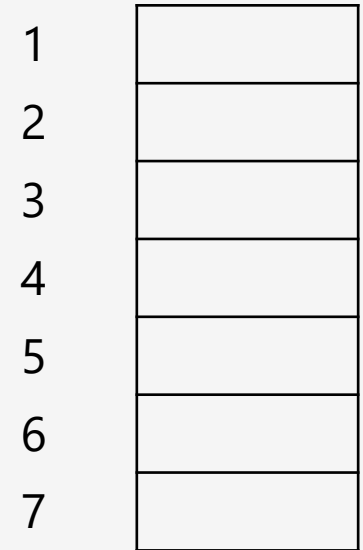
Un programme est une suite **d'instructions** élémentaires, qui vont être exécutées dans l'ordre par le **processeur**, Ces instructions correspondent à des **actions** très simples, comme **additionner** deux nombre, **lire** ou **écrire** une case mémoire, etc.

N.B: Chaque instruction est codifiée en mémoire sur quelques octets.

Généralité sur l'informatique

❑ Notion de mémoire principale

- La mémoire est divisée en emplacements de taille fixe (par exemple 8 bits), utilisés pour stocker les instructions et les données (Au moins temporairement).
- La mémoire de la machine est formée d'un grand nombre de cellules numérotées.
- Le numéro d'une case mémoire est appelé « adresse mémoire ».



Cases mémoire

Généralité sur l'informatique

❑ Notion de processeur

Est le cerveau de l'ordinateur. Il permet de manipuler des informations numériques, c'est-à-dire des informations codées sous forme binaire {0, 1}, et d'exécuter les instructions stockées en mémoire.

Pour chaque instruction, le processeur effectue schématiquement les opérations suivantes :

1. Lire en mémoire (MP) l'instruction à exécuter.
2. Effectuer le traitement correspondant.
3. Passer à l'instruction suivante.

❑ Notion de périphériques

Le programme reçoit des données des périphériques en entrée, et communique ses résultats en sortie à des périphériques. Une liste (non exhaustive) de périphériques usuels est :

- le clavier qui permet à l'utilisateur de saisir du texte ;
- la souris qui permet à l'utilisateur de sélectionner, d'activer ou de créer à la main des objets graphiques ;
- l'écran qui permet aux programmes d'afficher des données sous forme graphique ;
- l'imprimante qui permet de sortir des données sur support papier ;

Généralité

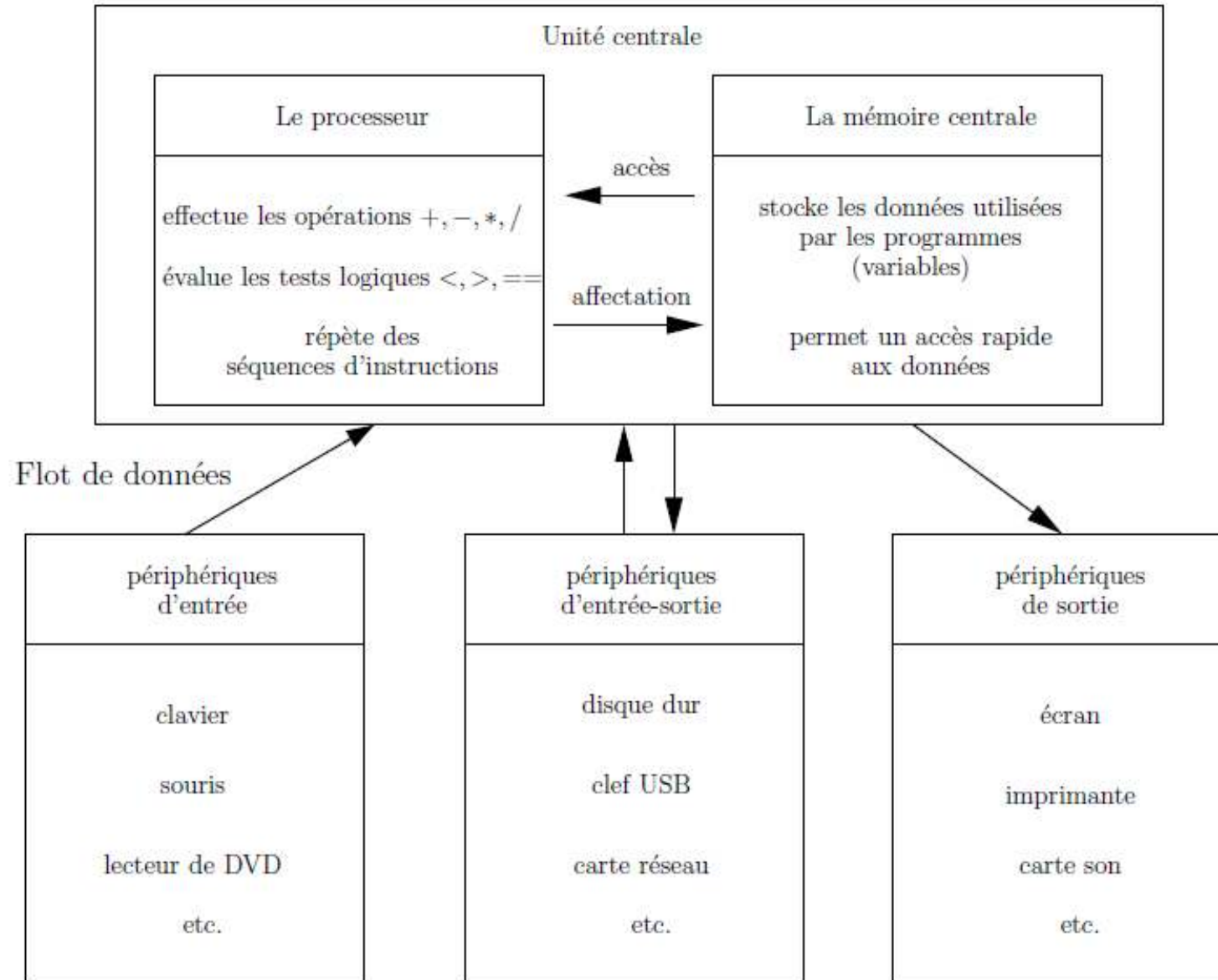


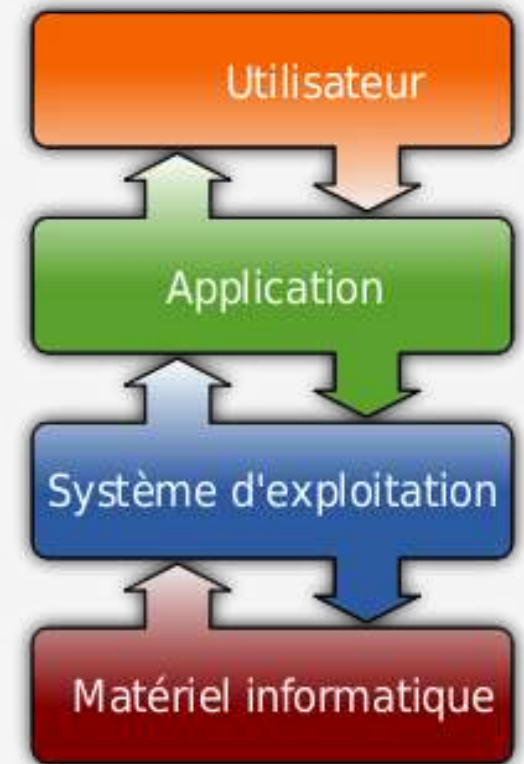
Figure 1.1- Schéma d'architecture d'un ordinateur.

Généralité sur l'informatique

❑ Notion de système d'exploitation

→ Un programme informatique doit recevoir des données pour les traiter, et produire d'autres données. Pour que le programme puisse fonctionner, il faut du matériel (composants électroniques), et il faut une couche logicielle **intermédiaire** avec le matériel, appelée système d'exploitation. Le système d'exploitation assure la communication entre le programme informatique et le matériel, et permet au programme **d'agir** sur le matériel.

→ Autre définition: Un système d'exploitation (OS: Operating System) est un ensemble de programmes qui **dirige** l'utilisation des ressources d'un ordinateur par des logiciels applicatifs.



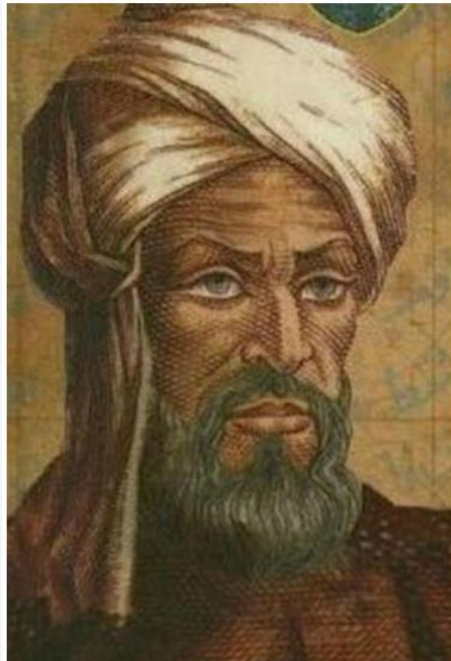
Partie 2: Algorithmique

Chapitre 1: Les éléments de base d'un algorithme

Algorithmique – Notion et définition d'un algorithme

D'où vient le mot **algorithme**?

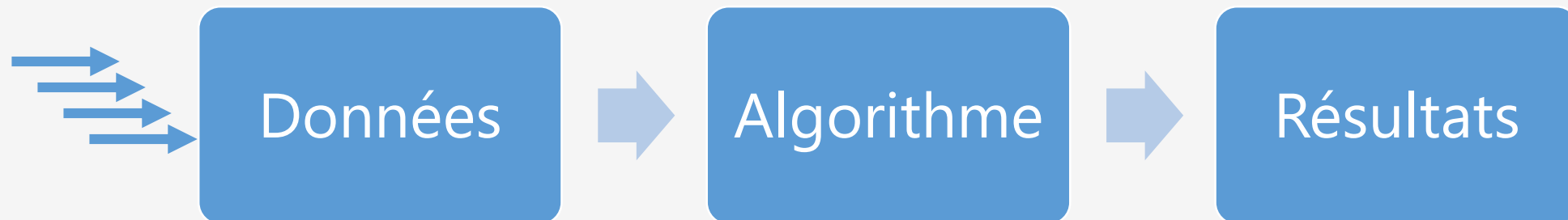
Muhammad Ibn Mûsâ al-Khuwârizmî (~ 780-850)
mathématicien, géographe, astrologue et astronome



Algorithmique – Notion et définition d'un algorithme

Le terme **algorithme** vient du nom du mathématicien Al-Khwârizmî. Il est employé en informatique pour décrire une **méthode de résolution de problème** programmable sur machine.

Un algorithme est la description de la méthode de **résolution** d'un **problème** donné en utilisant des **instructions** élémentaires permettant d'aboutir à un **résultat**. Ces instructions deviennent compréhensibles par l'ordinateur lors de **la traduction de l'algorithme en programme**.



Algorithmique – Notion et définition d'un algorithme

Exemple 1: (Algorithme de plantation d'un arbre)

Creuser un trou, reboucher un trou et placer un arbre dans un trou sont des opérations élémentaires (des actions simples) que toute personne (machine dans le cas de l'informatique) est censée savoir exécuter. Néanmoins, si un jardinier (programmeur) veut faire planter un arbre par une personne **qui ne sait pas le faire**, il doit lui fournir "un descriptif" (un algorithme) qui lui indique les opérations à faire ainsi que leur ordre d'exécution (séquencement).

----- Algorithme de plantation d'un arbre -----

1. **Creuser un trou**
2. **Placer l'arbre dans le trou**
3. **Reboucher le trou**

Algorithmique – Notion et définition d'un algorithme

Exemple 2 (Algorithme de plantation et d'arrosage de plusieurs arbres):

Pour planter et arroser un ensemble d'arbres, on peut procéder de la manière suivante: planter l'ensemble d'arbres et les arroser tous à la fin.

----- Algorithme de plantation et d'arrosage de plusieurs arbres -----

- 1. Creuser un trou.**
- 2. Placer un arbre dans le trou.**
- 3. Reboucher le trou.**
- 4. S'il existe encore des arbres exécuter les actions 1, 2, 3 et 4.
Sinon exécuter les actions suivantes.**
- 5. Arroser les arbres.**

Remarque:

Pour résoudre le problème précédent, on peut procéder autrement : planter et arroser arbre par arbre. On conclut alors qu'à un problème donné pourraient correspondre plusieurs algorithmes.

Algorithmique – Algorithmique et programmation

Tout problème à programmer doit être résolu, d'abord sous forme d'algorithme, puis converti en programme dans le langage de votre choix.

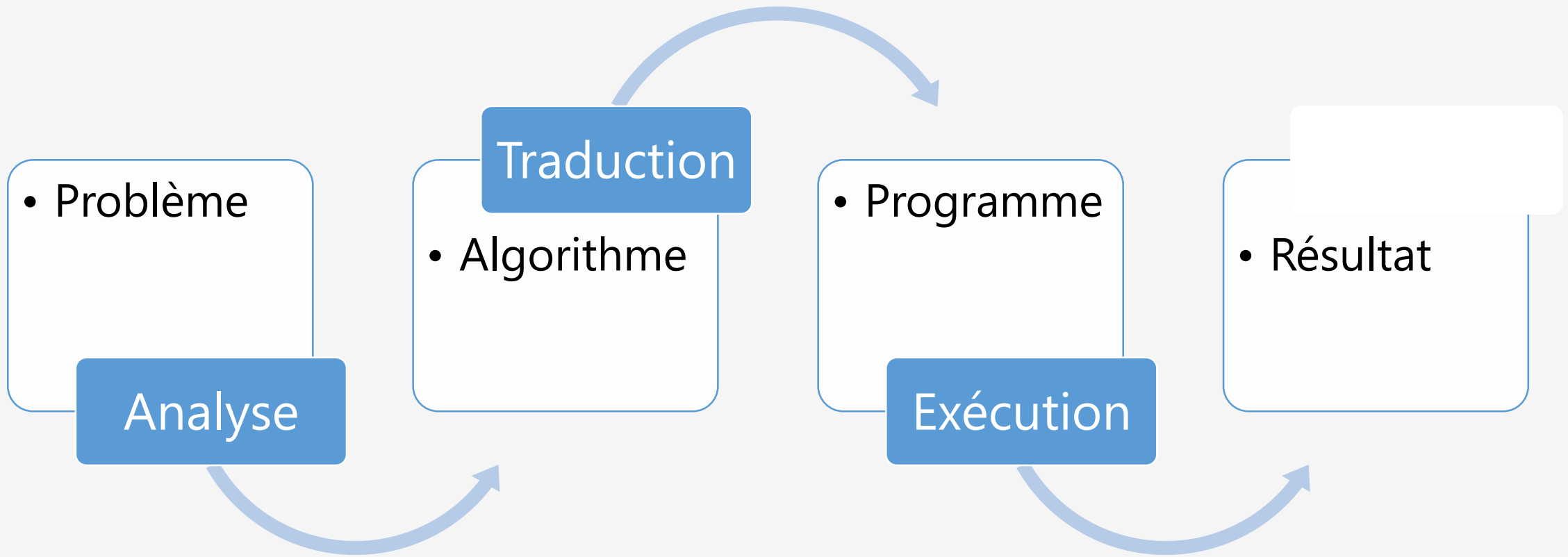
Un algorithme est indépendant du langage de programmation utilisé.

Programme:

Est un enchainement d'instructions, écrit dans **un langage de programmation**, exécutées par un ordinateur, permettant de traiter un problème et de renvoyer des résultats. Il présente **la traduction d'un algorithme** à l'aide d'un langage de programmation.

Algorithmique – Algorithmique et programmation

❑ Cycle de développement d'un programme informatique:



Algorithmique – Structure générale d'un algorithme

En général, un algorithme est composé de trois parties principales:

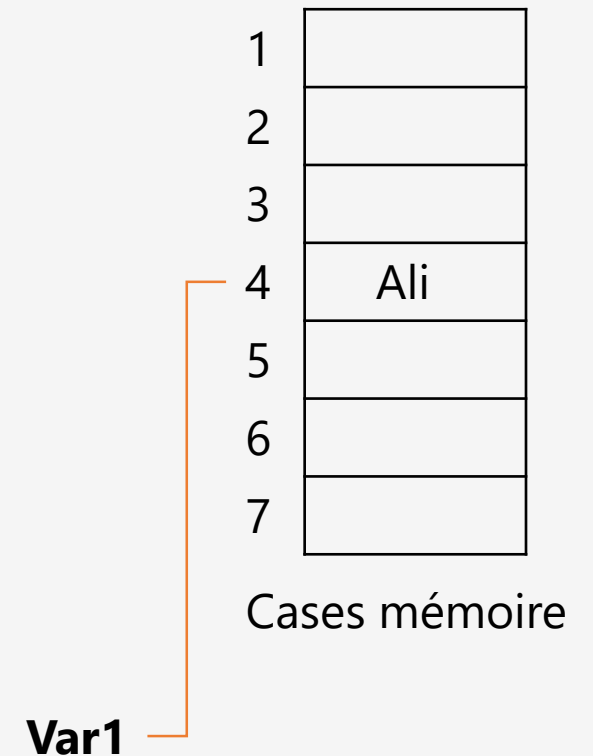
- ❑ L'en-tête: Cette partie sert à donner un nom à l'algorithme. Elle est précédée par le mot **Algorithme**.
- ❑ La partie déclarative: Dans cette partie, on déclare les différents objets que l'algorithme utilise (variables, constantes, ...)
- ❑ Le corps de l'algorithme: Cette partie contient les instructions (opérations) de l'algorithme. Elle est délimitée par les mots **Début** et **Fin**.

En-tête	{	Algorithme Nom de l'algorithme
Déclaration des variables et constantes	{	Variable Identificateur: type Constante Identificateur=valeur
Corps de l'algorithme (traitement)	{	Début Instruction 1 Instruction 2 Instruction n Fin

Algorithmique – Les variables et les constantes

❑ Notion de variable

- L'algorithme ne manipule pas directement les valeurs des données.
→ C'est pour cela que les données ainsi que les résultats des calculs intermédiaires ou finaux, sont organisées dans des cases mémoires qui correspondent à des **variables**.
- Une variable est associée à une zone de la mémoire de la machine appelé « case mémoire » ou « mot mémoire ».
- Elle est repérée par un nom, ou **identificateur**.
- Elle contient une **valeur** qui peut varier au cours de l'exécution de l'algorithme, d'où le nom de variable.
- Au départ la valeur est indéfinie
- Toutes les valeurs qu'elle peut prendre sont d'un même type, c'est à dire sont de même nature et peuvent subir les mêmes opérations de base.
- Quand une variable est déclarée dans un algorithme, la machine lui associe une adresse mémoire.
- Dans l'algorithme, on ne manipule que le nom des variables, pas l'adresse mémoire.



Algorithmique – Les variables et les constantes

□ Notion de variable

→ Une variable possède donc :

- ✓ Une valeur contenue par la case mémoire.
- ✓ Un identificateur: nom unique par lequel on peut accéder à son contenu.
- ✓ Un type qui définit la taille de la place occupée.

→ Ne pas confondre la variable et son contenu

- ✓ Une variable est un contenant(case ou boîte)
- ✓ Le contenu d'une variable est une valeur numérique, alphanumérique...

Algorithmique – Les variables et les constantes

❑ Déclaration des variables (1)

La partie déclaration consiste à **énumérer toutes les variables** dont on aura besoin au cours de l'algorithme.

Chaque déclaration doit comporter le **nom de la variable** (identificateur) et **son type**.

La syntaxe: → **Variable** identificateur : type

Exemple →

Variable X : entier

Variable age : réel

Variable a,b,c : entiers

Variable prenom : chaîne

Variable resultat : logique

Algorithmique – Les variables et les constantes

❑ Déclaration des variables (2)

- Identificateur

Un identificateur est le nom donné à une variable, une constante, fonction, etc. Ce nom doit obligatoirement **commencer** par une lettre suivie d'une suite de lettres et de chiffres et ne doit pas contenir d'espace.

Algorithmique – Les variables et les constantes

❑ Déclaration des variables (2)

- Types de données

Un type détermine en particulier les valeurs possibles de la variable et les opérations primitives applicables à cette variable.

Type de données	Numérique		Alphanumérique		Booléen
	Entier (Sans la virgule)	Réel (Avec et sans la virgule)	Caractères	Chaîne de caractères	
Exemple	-17 109 2020	34,17 -177,19 10	"A" "#" "? " "\$"	"2" "+" "al-Khuwarizmi", "Bonjour", "112020"	Deux valeurs possibles: Soit Vrai, soit Faux

Algorithmique – Les variables et les constantes

❑ Déclaration des variables (2)

- Types de données

Préciser le type des données suivantes:

Donnée	Type
-700	
"9"	
"Welcome to EST FBS"	
-40.99	
" # "	
Faux	
"Vrai"	

Algorithmique – Les variables et les constantes

❑ Déclaration des variables (2)

- Types de données

Préciser le type des données suivantes:

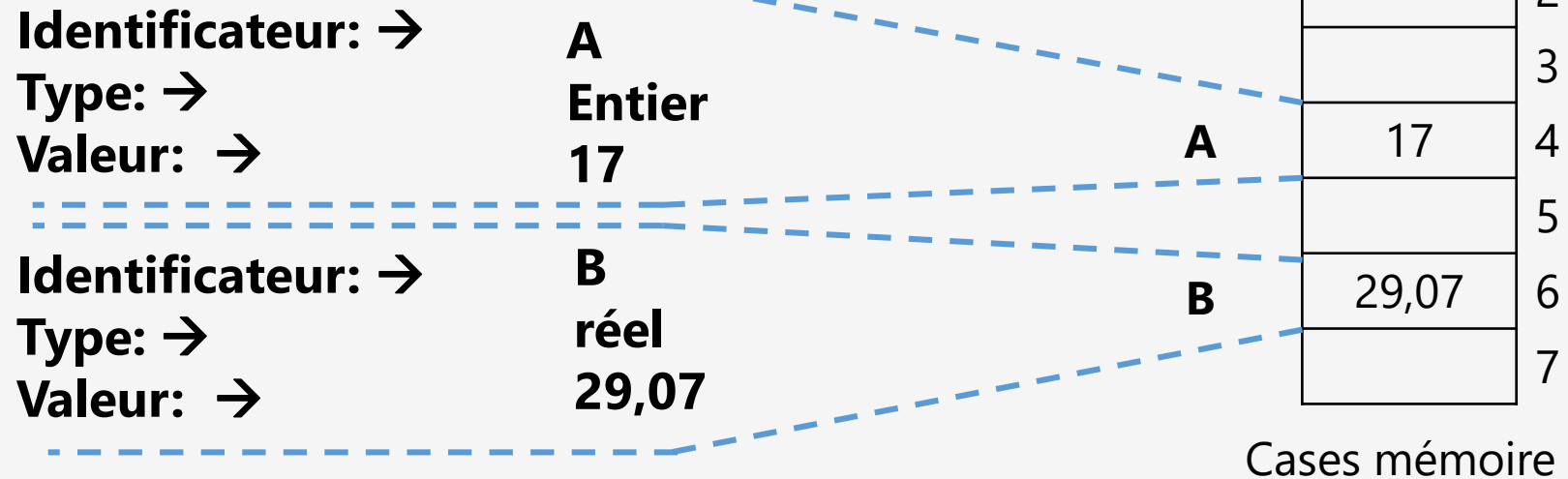
Donnée	Type
-700	entier
"9"	caractère
"Welcome to EST FBS"	Chaine de caractères
-40.99	réel
" # "	Chaine de caractères
Faux	booléen
"Vrai"	Chaine de caractère

Algorithme – Les variables et les constantes

❑ Déclaration des variables (3)

Déclarer une variable :

- réserver une place en mémoire
- attribuer l'identificateur à cette place



Algorithme – Les variables et les constantes

❑ Les Constantes

Une constante est un objet dont l'état reste inchangé durant toute l'exécution d'un programme. On ne peut jamais modifier sa valeur et celle-ci doit donc être précisée lors de la définition de l'objet.

Déclaration des constantes

Syntaxe: → Constante identificateur = valeur

Exemple → Constante PI = 3.14

Algorithme – Les instructions de base

Une instruction est une action élémentaire commandant à la machine un calcul. Les instructions de base sont:

1. L'instruction d'affectation: L'affectation, notée par le symbole $\leftarrow$, est l'opération qui évalue une expression (constante ou une expression arithmétique ou logique) et attribue la valeur obtenue à une variable.

Syntaxe: $\rightarrow$ Variable $\leftarrow$ expression

$a \leftarrow 17$ (a reçoit la valeur 17)

$a \leftarrow (a*b)+c$ (a reçoit le résultat de $(a*b)+c$)

Exemple d'affectation

$a \leftarrow 'K'$ (a reçoit la lettre K)

$a \leftarrow X \text{ et } Y$ (a reçoit le résultat de l'expression logique)

Algorithme – Les instructions de base

2. L'instruction d'entrée ou de lecture

Cette instruction permet d'attribuer à une variable une valeur introduite au moyen d'un organe d'entrée (généralement le clavier).

Syntaxe: → Lire (identificateur)

Exemple

Lire (A)

Lire (A,B,C)

Remarque: Lorsque le programme rencontre cette instruction, l'exécution s'interrompt et attend que l'utilisateur tape une valeur. Cette valeur est rangée en mémoire dans la variable désignée.

Algorithme – Les instructions de base

3. L'instruction de sortie ou de l'écriture

Cette instruction permet de communiquer une valeur donnée ou un résultat d'une expression à l'organe de sortie. En d'autres termes elle permet d'afficher des informations à l'écran.

Syntaxe: → **Ecrire** (expression)

expression: une valeur, un résultat, un message, etc

Exemple 1

Ecrire (A) (affiche à l'écran la valeur de la variable A)

Exemple 2

$A \leftarrow 17$

Ecrire ("La valeur de A est = ", A) Que affiche la dernière ligne ?

Algorithme – Les opérateurs et expressions

→ Les opérations possibles sur les variables **dépendent de leur type**.

- Exemple: On ne peut pas multiplier des mots

1. Les opérateurs de comparaison: $>$, $>=$, $<$, $<=$, $=$, $\neq$ (entiers, réels et caractères)

2. Les opérateurs arithmétiques:

Les opérateurs arithmétiques permettent de faire des calculs entre des valeurs numériques :

- Entre des réels: $+$, $-$, $*$ (la multiplication), $/$ (la division), $^$ (Exposant) + les opérateurs de comparaison.
- Entre deux entiers: $+$, $-$, $*$ (la multiplication), $/$ (la division), $^$ (Exposant), **DIV** (la division entière), **MOD** (le reste de la division entière) + les opérateurs de comparaison.

3. Les opérateurs de concaténations: Entre chaînes: Les opérateurs de comparaison, la concaténation (**&**)

4. Les opérateurs logiques: **ET** , **OU** , **NON**

Algorithme – Exemple d'algorithme

1. Nom d'algorithme

2. Déclaration de variables

3. Corps de l'algorithme

```
Algorithme nbr_poissons
Variable nbr_poiss : entier
           poids_t, poids_poiss : réel
Debut
poid_t <-- 0
nbr_poiss <-- 0
TantQue poid_t < 1000
Ecrire ('Donner le poids du poisson pesé')
Lire (poids_poiss)
poid_t <-- poid_t + poids_poiss
nbr_poiss <-- nbr_poiss + 1
FinTantQue
Ecrire ('Le nombre de poisson est:', nbr_poiss)
Fin
```

Algorithme – Exemple d'algorithme

Algorithme – Exercice

Exercice 1: Calcule PTTC

Ecrire un algorithme qui permet de saisir le prix HT (PHT) d'un produit et de calculer son prix total TTC (PTTC), TVA = 20%.

Algorithme – Exercice

Solution:

Ecrire un algorithme qui permet de saisir le prix HT (PHT) d'un produit et de calculer son prix total TTC (PTTC), TVA = 20%.

```
Algorithme Calcul_PTTC
Variables PHT, PTTC : réel
Constante TVA = 0.2
Début
    Ecrire ("Entrer le prix hors taxes:")
    Lire (PHT)
     $PTTC \leftarrow PHT + (PHT * TVA)$ 
    Ecrire ("Le prix TTC est =", PTTC)
Fin
```

Algorithme – Exercice

Exercice 2: **Ecrire un algorithme qui demande à l'utilisateur de saisir le rayon r et la hauteur h d'un cylindre et qui permet d'afficher sa surface totale et son volume.**

$$\text{Surface} = 2\pi r(h+r)$$

$$\text{Volume} = \pi r^2 h$$

Algorithme – Exercice

Solution :

Ecrire un algorithme qui permet de calculer le périmètre et la surface d'un cercle

```
Algorithme Cercle
Variables Ray, Haut, Volu, Surf : réel
Constante P = 3.14
Début
    Ecrire ("Entrer le rayon:")
    Lire (Ray)
    Ecrire ("Entrer la hauteur:")
    Lire (Haut)
    Volu ← P*Ray*Ray*Haut
    Surf ← 2*P*Ray(Haut+Ray)
    Ecrire ("La surface est =", Surf)
    Ecrire ("Le Volume est =", Volu)
Fin
```


Algorithme – Exercice

Exercice 3: Écrire un programme qui permute la valeur de deux variables c1 et c2 de type caractère

Algorithme – Exercice

Solution

Algorithme Permutation_deux_caractères

Variables c1, c2 : caractère

Variable inter : caractère

Début

Ecrire ("Entrer le premier caractère:")

Lire (c1)

Ecrire ("Entrer le deuxième caractère:")

Lire (c2)

inter $\leftarrow$ c1

c1 $\leftarrow$ c2

c2 $\leftarrow$ inter

Ecrire ("Les valeurs de c1 et c2 après la permutation sont:", c1,c2)

Fin

Partie 2: Algorithmique

Chapitre 2: Les structures conditionnelles et répétitives

A- Les structures conditionnelles

A- Les structures conditionnelles (Alternatives)

Introduction:

Traitement séquentiel # Structure Conditionnelle

Un traitement séquentiel contient une procédure étape par étape à exécuter, alors que la structure conditionnelle ou alternative permet d'exécuter ou non une série d'instructions selon la valeur d'une condition.

A- Les structures conditionnelles (Alternatives)

1. La structure Si ... Alors ... FinSi

Syntaxe:

Si condition **Alors**

 traitement *-- une séquence d'instructions*

FinSi

Règle : La condition est nécessairement une expression ou une variable booléenne.

Évaluation :

1. la condition est évaluée ;
2. si la condition est **vraie**, le **traitement** est exécuté puis le contrôle passe à l'instruction qui suit le **FinSi** ;
3. si la condition est **fausse**, la machine saute directement à l'instruction qui suit le **FinSi**.
En d'autres termes, le traitement est effectué si et seulement si la condition est **VRAI**.

A- Les structures conditionnelles (Alternatives)

Exercice 1:

Écrire un algorithme qui lit une valeur entière au clavier et affiche « pair » si elle est pair.

A- Les structures conditionnelles (Alternatives)

Solution

```
Algorithme Nombre_pair
Variables Nbr, R : entier

Début
  Ecrire ("Entrer un nombre:")
  Lire (Nbr)
  R ← Nbr MOD 2
  Si R=0 Alors
    Ecrire (Nbr," est pair")
  FinSi
Fin
```

A- Les structures conditionnelles (Alternatives)

2. La structure Si ... Alors ... Sinon ... FinSi

Syntaxe:
Si condition **Alors**
 Instruction(s) 1
Sinon
 Instruction(s) 2
FinSi

Règle : La condition est nécessairement une expression ou une variable booléenne.

Évaluation :

1. la condition est évaluée ;
2. si la condition est **vraie**, la série **d'instructions 1** est exécutée et l'ensemble d'instructions 2 est ignoré. puis le contrôle passe à l'instruction qui suit le **FinSi** ;
3. De même, si la condition est **fausse**, la machine saute directement à la première ligne située après le **Sinon** et exécute l'ensemble **d'instruction2**.

A- Les structures conditionnelles (Alternatives)

Exercice 2:

Écrire un algorithme qui lit un nombre entier au clavier et affiche s'il s'agit d'un nombre positif ou négatif.

A- Les structures conditionnelles (Alternatives)

3. La structure Si ... Alors ... SinonSi ... Sinon ... FinSi

Syntaxe:

Si **condition1** **Alors**

Instruction(s) 1

SinonSi **condition2** **Alors**

Instruction(s) 2

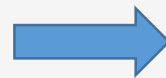
SinonSi **condition3** **Alors**

Instruction(s) 3

Sinon

Instructions

FinSi



L'instruction qui sera exécutée est l'instruction dont la condition est vraie.

Si aucune condition n'a la valeur vraie, c'est l'instruction qui suit la **Sinon** qui sera exécutée.

A- Les structures conditionnelles (Alternatives)

Exercice 3:

Écrire un algorithme qui demande le numéro d'un mois et affiche le nombre de jours que comporte ce mois en utilisant la structure Si ... Alors ... SinonSi ... Sinon ... FinSi.

Janvier, Mars, Mai, Juillet, Août, Octobre, Décembre → **31 jours**

Avril, Juin, Septembre, Novembre → **30 jours**

Février → **28 jours**

A- Les structures conditionnelles (Alternatives)

Algorithme Nombre_de_jours_par_Mois

Variable mois: entier

Début

Ecrire("Entrer une valeur entre 1 et 12")

Lire(mois)

Si(mois=1 ou mois=3 ou mois=5 ou mois=7 ou mois=8 ou mois=10 ou mois=12) **Alors**

Ecrire(« Ce mois comporte 31 jours")

SinonSi (mois=4 ou mois= ou mois=6 ou mois=9) **Alors**

Ecrire("Ce mois comporte 30 jours")

SinonSi (mois=2) **Alors**

Ecrire("Ce mois comporte 28 jours")

Sinon

Ecrire (« Nombre invalide, ce mois n'existe pas")

Fin

A- Les structures conditionnelles (Alternatives)

Exercice 4:

**Ecrire un algorithme qui permet d'afficher le maximum parmi deux nombres saisis au clavier.
(Utiliser l'instruction **Si ... Alors ... SinonSi ... Sinon ... FinSi**).**

A- Les structures conditionnelles (Alternatives)

Algorithme Maximum_de_deux_nombres

Variable A,B: réel

Début

Ecrire("Entrer la valeur de A")

Lire(A)

Ecrire("Entrer la valeur de B")

Lire(B)

Si(A > B) **Alors**

Ecrire("Le maximum est", A)

SinonSi (A = B) **Alors**

Ecrire("Egalité")

Sinon

Ecrire ("Le maximum est", B)

FinSi

Fin

A- Les structures conditionnelles (Alternatives)

Remarque:

Une structure conditionnelle peut contenir à son tour une autre structure conditionnelle, c'est-à-dire l'une incluse dans l'autre

Exemple :

Ecrire un algorithme permettant de calculer le montant des allocations familiales sachant que le montant dépend du nombre d'enfants:

- Si nombre d'enfants est inférieur ou égal à 3 alors les allocations sont de 300 dh par enfant.
- Si le nombre d'enfants est strictement supérieur à trois et inférieur ou égal à 6 alors les allocations sont de:
 - 300 dh par enfant pour les trois premiers enfants
 - 100 dh par enfant pour les suivants
- Si le nombre d'enfants est strictement supérieur à 6 alors les allocations sont:
 - 300 dh par enfant pour les trois premiers enfants
 - 100 dh par enfant pour les suivants
 - 0 dh par enfant pour les suivants

A- Les structures conditionnelles (Alternatives)

Algorithme Allocations_familiales

Variable NbrEnf, Montant: entier

/ NbrEnf: Nombre d'enfants, Montant: Montant des allocations familiales */*

Début

Ecrire("Entrer le nombre d'enfants")

Lire(NbrEnf)

Si (NbrEnf ≤ 3) **Alors**

Montant ← NbrEnf * 300

Sinon

Si (NbrEnf ≤ 6) **Alors**

Montant ← (3*300) + (NbrEnf - 3) * 100

Sinon

Montant ← 1200

FinSi

FinSi

Ecrire("Le montant d'allocations familiales est", Montant)

Fin

A- Les structures conditionnelles (Alternatives)

4. La structure à choix multiples

Cette instruction conditionnelle permet de **choisir** le traitement à effectuer **en fonction de la valeur** ou de **l'intervalle de valeurs** d'une variable ou d'une expression.

Syntaxe:

```
Selon expression faire  
    choix 1: action(s)1  
    choix 2: action(s)2  
    choix 3: action(s)3  
    ...  
    choix n: action(s)n  
Sinon  
    actions  
FinSelon
```

A- Les structures conditionnelles (Alternatives)

4. La structure à choix multiples

- Evaluation:

- ✓ L'expression est évaluée, puis sa valeur est successivement comparée à chacun des ensembles choix i .
- ✓ Dès qu'il y a correspondance, les comparaisons sont arrêtées et la séquence associée est exécutée.
- ✓ À la fin de l'exécution, on continuera la suite de l'exécution du programme à partir de la 1^{ère} instruction qui vient après **FinSelon**.
- ✓ Si aucun choix ne correspondant, on exécutera le traitement **Sinon** du **Selon**.

Syntaxe:

Selon expression **faire**
choix 1: action(s)1
choix 2: action(s)2
choix 3: action(s)3
...
choix n: action(s)n

Sinon
actions
FinSelon

A- Les structures conditionnelles (Alternatives)

4. La structure à choix multiples

- Remarques:

1. Le sélecteur doit être de type **scalaire discret** et doit avoir une valeur avant d'être impliqué dans le **Selon**.
2. Les traitements relatifs aux valeurs peuvent comporter également d'autres structures conditionnelles.
3. Choix i peut être un ensemble des valeurs séparées par des virgules ou intervalle des valeurs ($V_i .. V_f$).
4. Si la valeur du sélecteur est différente des valeurs proposées alors c'est le traitement qui suit la clause **sinon** qui sera exécuté.

Syntaxe:

```
Selon expression faire  
    choix 1: action(s)1  
    choix 2: action(s)2  
    choix 3: action(s)3  
    ...  
    choix n: action(s)n  
Sinon  
    actions  
FinSelon
```

A- Les structures conditionnelles (Alternatives)

4. La structure à choix multiples

Exemple:

Ecrire un algorithme permettant d'afficher le jour en toute lettre selon son numéro saisi au clavier.

Algorithme Jours

Variable J: entier

Début

Ecrire("Entrer le numéro du jours")

Lire(J)

Selon (J) Faire

1: Ecrire("Dimanche")

2: Ecrire("Lundi")

3: Ecrire("Mardi")

...

7: Ecrire("Samedi")

Sinon

Ecrire ("Le numéro saisi ne correspond pas à un jours")

FinSelon

Fin

A- Les structures conditionnelles (Alternatives)

Exercice 5:

Ecrire un algorithme qui vérifie si le caractère saisi par l'utilisateur est une voyelle ou une consonne en utilisant **Selon**.

Les voyelles sont: a, e, i, o, u, y

A- Les structures conditionnelles (Alternatives)

Solution:

Partie 3: Notions de base en programmation avec Quick Basic

Partie 3: Notions de base en programmation avec Quick Basic

Définition d'un programme :

- ❑ Un programme est une suite d'instructions données préalablement par le programmeur à la machine. Cette suite sera ultérieurement exécutée par l'ordinateur qui affichera alors des choses à l'écran, demandera à l'utilisateur du programme des données, fera des calculs, réagira lorsque l'utilisateur appuiera sur les touches du clavier, etc...
- ❑ Un compilateur est un programme qui traduit un texte écrit dans un langage de programmation de haut niveau vers un code binaire prêt pour exécution par l'ordinateur.

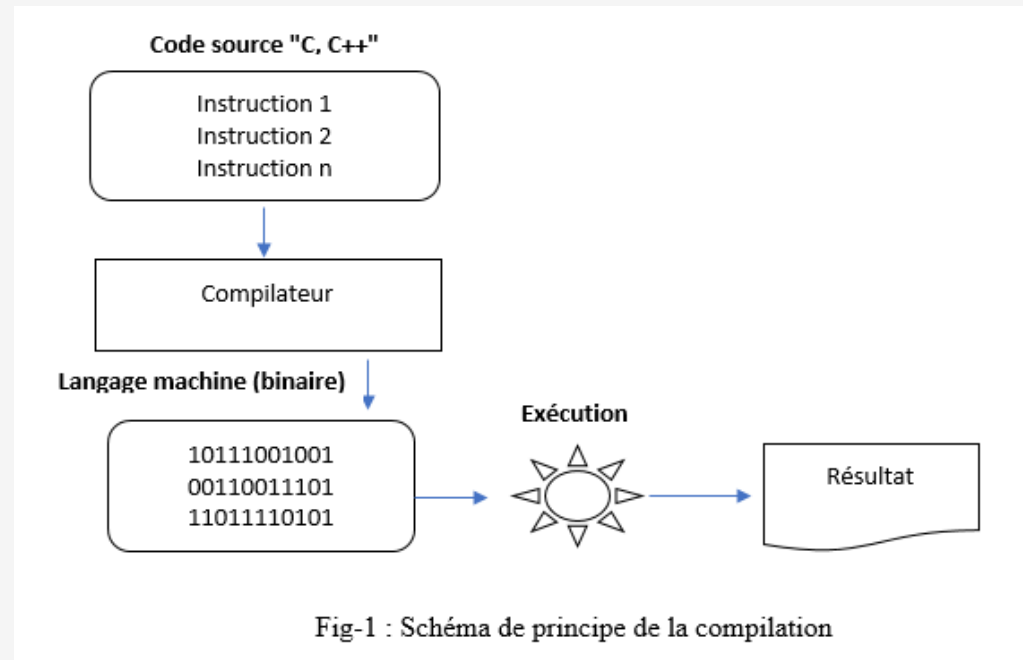


Fig-1 : Schéma de principe de la compilation

Concepts fondamentaux de programmation

1. Les variables:

❑ **Définition:** Les données sont stockées dans des variables. Ces variables sont des emplacements dans la mémoire de l'ordinateur. Elles ont un nom et un type (fixes) elles contiennent une valeur qui elle peut varier.

- Le nom d'une variable permet de retrouver son emplacement en mémoire. Il équivaut à son adresse.
- Le type de la variable détermine **l'espace à lui réserver en mémoire** et **les traitements qui lui sont appropriés**.

❑ **Types de variables:**

	Nom du type	Taille	Min et max
Entier	INTEGER	2 octets	-32 768 à + 32 767
	LONG	4 octets	- 2 147 483 648 à + 2 147 483 647
Réel	SINGLE	4 octets	$\pm 1.40 \cdot 10^{-45}$ à $\pm 3.40 \cdot 10^{+38}$
	DOUBLE	8 octets	$\pm 4.94 \cdot 10^{-324}$ à $\pm 1.79 \cdot 10^{+308}$
Chaîne de caractères	STRING		

Concepts fondamentaux de programmation

1. Les variables (suite):

- ❑ **Déclaration de variables:** Déclarer une variable, c'est lui réserver un emplacement en mémoire auquel on donne un nom et en indiquant son type

Syntaxe: **DIM** NomVariable **AS** type

Exemple: **DIM** Age **AS** INTEGER
DIM Prenom **AS** STRING

- ❑ **Affectation:** Assigner une valeur à une variable, c'est écrire cette valeur dans l'emplacement mémoire réservé à cette variable.

Exemple: Age = 19
Prenom = "Ali"

NB : La déclaration des variables est facultative en Quick Basic.

Concepts fondamentaux de programmation

2. Entrées / Sorties

❑ Les instructions PRINT et CLS :

Pour afficher du texte à l'écran, il faut utiliser l'instruction **PRINT** en plaçant le texte entre des guillemets.

CLS (Clear screen) est utilisée pour effacer l'affichage.

Programme	Résultat du programme
PRINT "Salut les amis"	Salut les amis
PRINT " 3+4 "	3+4
PRINT	
PRINT 3+4	7
PRINT "Bonjour";	BonjourAhmed
PRINT "Ahmed"	
PRINT "Bonjour",	Bonjour Ahmed
PRINT "Ahmed"	

Concepts fondamentaux de programmation

2. Entrées / Sorties

❑ L'instruction INPUT (Comment permettre à l'utilisateur d'entrer des données dans le programme ?)

Le rôle de cette instruction est d'inviter l'utilisateur à entrer une donnée dans le programme qui sera stockée par la suite dans une variable précise.

Exemple:

PRINT "Quel est ton nom ?"

INPUT Nom

Programme	Résultat du programme
PRINT "Donner le montant en Euros:" INPUT M PRINT "La valeur en dh est:" ; M*10.80	Donner le montant en Euros: ? 2 La valeur en dh est: 21,6

Concepts fondamentaux de programmation

3. Les opérateurs

A- Les opérateurs de comparaison

Opérateur	Description
=	Comparaison d'un égalité
<>	Comparaison d'une différence
>	Comparaison de plus grand que
>=	Comparaison de plus grand ou égal que
<	Comparaison de plus petit que
<=	Comparaison de plus petit ou égal que

B- Les opérateurs arithmétiques

Opérateur	Description
+	Effectue une addition
-	Effectue une soustraction
*	Effectue une multiplication
/	Effectue une division réel
\	Effectue une division entière
<u>MOD</u>	Effectue le reste d'une division (modulo)
^	Effectue une opération d'exposant
<u>AND</u>	Effectuer un «Et binaire»
<u>NOT</u>	Effectue une «Négation binaire»
OR	Effectuer un «Ou binaire»

Concepts fondamentaux de programmation

3. Les structures conditionnelles

→ **Comment faire en sorte que le programme réagisse différemment en fonction de ce que l'utilisateur aura tapé ?**

1. Les instructions **IF ... THEN ... END IF** et **IF ... THEN ... ELSEIF ... ELSE ... END IF**:

Ces structures permettent d'exécuter ou bien non un bloc d'instructions selon le résultat d'un test de comparaison.

Programme	Résultat du programme
CLS PRINT "Quelle est votre âge " INPUT age IF age >= 80 THEN PRINT "Vous n'êtes plus jeune." END IF IF age <= 20 THEN PRINT "Gamin !" END IF IF age > 20 AND age < 80 THEN PRINT "Ni jeune ni vieux!" END IF	Quelle est votre âge ? 5 Gamin !

Programme	Résultat du programme
CLS PRINT "Quelle est votre âge " INPUT age IF age >= 80 THEN PRINT "Vous n'êtes plus jeune." ELSEIF age <= 20 THEN PRINT "Gamin !" ELSE PRINT "Ni jeune ni vieux !" END IF	Quelle est votre âge ? 5 Gamin !

Concepts fondamentaux de programmation

3. Les structures conditionnelles

→ **Comment faire en sorte que le programme réagisse différemment en fonction de ce que l'utilisateur aura tapé ?**

2. L'instruction **SELECT CASE (SELON en algorithmique)**

L'instruction SELECT CASE convient mieux dès qu'il y a plus de trois conditions à tester

Programme	
SELECT CASE (<i>expression ou variable</i>)	Les listes de valeurs peuvent se présenter
CASE liste_de_valeurs_1 Séquence d'instruction 1	comme suit :
CASE liste_de_valeurs_1 Séquence d'instruction 2	2 = une seule valeur
...	12, 15, 21 = plusieurs valeurs
CASE ELSE Séquence d'instruction N	10 TO 20 = une plage de valeurs
END SELECT	IS > 12 = au-delà d'une valeur

```
1 DIM j AS INTEGER
2 PRINT "Entrer le numero de jour"
3 INPUT j
4 SELECT CASE j
5     CASE 1
6         PRINT "LUNDI"
7     CASE 2
8         PRINT "MARDI"
9     CASE 3
10        PRINT "MERCREDI"
11    CASE 4
12        PRINT "JEUDI"
13    CASE 5
14        PRINT "VENDREDI"
15    CASE 6
16        PRINT "SAMEDI"
17    CASE 7
18        PRINT "DIMANCHE"
19    CASE ELSE
20        PRINT "Numero de jours invalide"
21 END SELECT
```


Equivalence langage algorithmique / QBasic

Algorithme	Qbasic
Variable Nbr : entier Variable Nom : chaine de caractères	DIM Nbr AS INTEGER DIM Nom AS STRING
Nbr <-- 4 Nom <-- "Ali"	Nbr = 4 Nom = "Ali"
Ecrire (" Merci d'entrer votre nom")	PRINT "Merci d'entrer votre nom"
Lire (Nom)	INPUT Nom
Si Note > 18 Alors Ecrire ("Excellent travail")	IF Note > 18 THEN PRINT "Excellent travail"
Si Note > 18 Alors Ecrire ("Excellent travail") Sinonsi Note < 10 Alors Ecrire ("Insuffisant") FinSi	IF Note > 18 THEN PRINT "Excellent travail" ELSEIF Note < 10 THEN PRINT "Insuffisant" END IF
SELON <i>expression</i> Faire FinSelon	SELECT CASE <i>expression</i> END SELECT